

M12 Module Review

Distributed Processing with Spark / PySpark | post-attempt review

Open only after genuine attempt

Use this to diagnose reasoning, not to replace the first attempt.

Review L01 - Why Spark exists: driver, executors, partitions and tasks

Expected reasoning: You can trace one simple action from driver plan to partition tasks.

Repair trigger: Treating tasks as rows or thinking local mode teaches cluster sizing.

Review L02 - PySpark setup, SparkSession and laptop-safe local mode

Expected reasoning: You can verify the environment and explain the local profile limits.

Repair trigger: Running every heavy stack simultaneously or treating 1g/4 partitions as production defaults.

Review L03 - DataFrames and explicit schemas

Expected reasoning: Schema and grain are both explicit and validated.

Repair trigger: Relying on inference for ambiguous production inputs or losing grain.

Review L04 - Spark SQL and expression equivalence

Expected reasoning: You can explain the same transformation in SQL and DataFrame form.

Repair trigger: Writing two versions that differ in filter/grain and calling them equivalent.

Review L05 - Lazy transformations and actions

Expected reasoning: You can predict which line triggers execution.

Repair trigger: Assuming a transformation has already read/written all data.

Review L06 - Jobs, stages, tasks and physical plans

Expected reasoning: You can connect a plan operator to execution behavior.

Repair trigger: Inventing performance claims without plan/UI evidence.

Review L07 - repartition vs coalesce

Expected reasoning: You can choose a method based on purpose and scale.

Repair trigger: Using coalesce(1) on large production output because one file looks tidy.

Review L08 - Aggregations and shuffle cost

Expected reasoning: You can explain why the shuffle occurs and prove output correctness.

Repair trigger: Optimizing away a shuffle by changing the business result.

Review L09 - Joins and broadcast strategy

Expected reasoning: You can defend key uniqueness and broadcast choice.

Repair trigger: Broadcasting a huge fact table or ignoring duplicate dimension keys.

Review L10 - Join correctness, fan-out and reconciliation

Expected reasoning: You can prove the join preserves intended business identity.

Repair trigger: Treating a row-count increase as harmless without explaining grain.

Review L11 - Data skew

Expected reasoning: You can distinguish skew from general slowness.

Repair trigger: Assuming more partitions automatically fixes a hot key.

Review L12 - Adaptive Query Execution (AQE)

Expected reasoning: You can describe AQE as adaptive execution, not correctness insurance.

Repair trigger: Claiming AQE removes all need for partition/join reasoning.

Review L13 - Caching and persistence

Expected reasoning: You can justify cache from reuse and resource evidence.

Repair trigger: Caching every DataFrame by default.

Review L14 - Parquet I/O and partition pruning

Expected reasoning: You can explain file layout and read plan together.

Repair trigger: Equating partition pruning with row-level filtering or overpartitioning output.

Review L15 - Built-in functions vs Python UDFs

Expected reasoning: You can state when a UDF is justified.

Repair trigger: Using a Python UDF for simple arithmetic/string/date operations.

Review L16 - Driver safety: collect, toPandas and bounded inspection

Expected reasoning: You can explain where data moves and why the driver remains safe.

Repair trigger: Unbounded collect()/toPandas on unknown data size.

Review L17 - Testing and debugging Spark transformations

Expected reasoning: A clean test proves transformation behavior independently of file I/O.

Repair trigger: Only testing by eyeballing a large output file.

Review L18 - Integrated PySpark pipeline and distributed-processing checkpoint

Expected reasoning: Outputs reconcile and you can explain how Spark executed the key transformations.

Repair trigger: Calling the pipeline complete because it runs once without reconciliation.